

Decision Trees in Python

A Beginner's Step-by-Step Teaching Guide

Duration: 15–20 Minutes | Level: Absolute Beginner | Topic: Machine Learning

What Will We Learn Today?

By the end of this session, students will be able to:

- Understand what a Decision Tree is and how it works in plain English
- Read and understand every line of Python code in the notebook
- Know what each library does and why we use it
- Load data, clean it, and prepare it for machine learning
- Train a Decision Tree model and evaluate its accuracy
- Visualize the decision tree and interpret its rules

Section 1: What Is a Decision Tree?

Imagine you are a doctor. A new patient walks in. To decide which medicine to give them, you ask a series of YES/NO questions:

Real-Life Example — The Doctor's Thought Process:

Q1: Is the Na_to_K ratio greater than 14.6?

→ YES → Give Drug Y

→ NO → Ask next question...

Q2: Is the Blood Pressure HIGH?

→ YES → Ask the next question...

Q3: Is the Age more than 50?

→ YES → Give Drug B

→ NO → Give Drug A

This is EXACTLY how a Decision Tree works! It asks a series of questions about the data, and at the end of each path, it gives you a final answer (called a 'leaf node'). The computer learns WHICH questions to ask and in WHAT ORDER by looking at training data.

Key Terms to Remember:

Root Node → The very first question the tree asks (top of the tree)

Branch → The YES or NO path you take after answering a question

Leaf Node → The final answer at the bottom (e.g., Drug Y)

Depth → How many levels of questions the tree has

Entropy → A measure of how mixed up the data is at each step

Section 2: The Drug Prediction Dataset

In this notebook, we work with a real-world medical dataset. Here is the story behind it:

The Dataset Story:

200 patients all had the same illness.

Each patient was given one of 5 drugs: Drug A, Drug B, Drug C, Drug X, or Drug Y.

For each patient, we know: Age, Sex, Blood Pressure (BP), Cholesterol, and Na_to_K ratio.

Our goal: Given a NEW patient's details, predict which drug they should receive.

This is a classification problem — we are classifying patients into one of 5 drug categories. The Decision Tree will learn the rules automatically from the 200 rows of data.

Section 3: Importing Libraries — Line by Line

Before writing any code, Python needs to know which 'toolboxes' to use. We call these toolboxes libraries. Think of it like this: before cooking, you take out the pots, pans, and spices you need.

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn import metrics

%matplotlib inline

import warnings
warnings.filterwarnings('ignore')
```

Line 1: import numpy as np

Syntax:

```
import <library_name> as <short_name>
```

What it does: NumPy is a library for doing maths with numbers in Python. It is especially good at working with lists of numbers (called arrays).

Why 'as np'? Instead of typing 'numpy' every time, we give it a short nickname 'np'. It saves typing.

Example of NumPy in action:

```
import numpy as np
scores = np.array([85, 90, 78, 95, 88])
print(scores.mean()) # Output: 87.2
```

Just like in Excel, `np.mean()` calculates the average automatically!

Line 2: import pandas as pd

What it does: Pandas is the library for working with tables of data — exactly like an Excel spreadsheet. It lets you load CSV files, filter rows, and clean data easily.

Why 'as pd'? Again, just a short nickname for convenience.

Example of Pandas in action:

```
import pandas as pd
data = pd.read_csv('students.csv') # Load a file
print(data.head())                 # Show first 5 rows
print(data['Age'].mean())           # Average of Age column
```

Line 3: from matplotlib import pyplot as plt

Syntax:

```
from <library> import <specific_part> as <nickname>
```

What it does: Matplotlib is the drawing library. It creates charts and graphs. pyplot is the specific part of matplotlib that handles most of the plotting.

Example of Matplotlib in action:

```
import matplotlib.pyplot as plt
plt.bar(['Drug A', 'Drug B'], [50, 75]) # Draw a bar chart
plt.title('Drug Count')
plt.show()                               # Display the chart
```

Line 4: from sklearn.preprocessing import LabelEncoder

What it does: Scikit-learn (sklearn) is the main machine learning library in Python. LabelEncoder is a tool inside it that converts text labels into numbers.

Why do we need this? Decision Trees and most ML algorithms work with NUMBERS only. Words like 'Male', 'Female', 'High', 'Low' must be converted to 0, 1, 2, etc.

Example of LabelEncoder in action:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
colors = ['Red', 'Blue', 'Green', 'Blue', 'Red']
encoded = le.fit_transform(colors)
```

```
print(encoded) # Output: [2, 0, 1, 0, 2]
```

Red → 2, Blue → 0, Green → 1 (sorted alphabetically)

Line 5: `from sklearn.model_selection import train_test_split`

What it does: This function splits our dataset into two parts — a Training set and a Testing set.

Why do we split data?

Training set → The model **LEARNS** from this data (like studying notes before an exam)

Testing set → We **TEST** the model on this data it has never seen (like the actual exam)

We do this to check: Did the model really learn? Or did it just memorize?

In our notebook: 70% of data = training, 30% = testing

Line 6: `from sklearn.tree import DecisionTreeClassifier, plot_tree`

DecisionTreeClassifier: This is the actual Decision Tree algorithm. We use it to build and train our model.

plot_tree: This creates a visual diagram of the decision tree so we can see all the questions it asks.

Line 7: `from sklearn import metrics`

What it does: The metrics module gives us tools to measure how good our model is. For example, `accuracy_score` tells us what percentage of predictions were correct.

Line 8: `%matplotlib inline`

What it does: This is a special Jupyter Notebook command (called a 'magic command'). It tells Jupyter to show charts directly inside the notebook, below the code cell, instead of opening a separate window.

Note: This only works in Jupyter Notebook or JupyterLab. It does NOT work in regular Python files.

Lines 9-10: `import warnings / warnings.filterwarnings('ignore')`

What it does: Sometimes Python prints warning messages that are not errors but are noisy and confusing for beginners. These two lines simply hide those warning messages so the output stays clean.

Section 4: Loading and Understanding the Data

```
path = 'https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/.../drug200.csv'
my_data = pd.read_csv(path)
my_data
```



Syntax:

`pd.read_csv( url_or_filepath )` → Loads a CSV file into a pandas table (called a DataFrame)

What happens here: We load a CSV file from the internet directly into a pandas DataFrame called 'my_data'. A DataFrame is just a table with rows and columns, exactly like an Excel sheet.

Example — Loading a local file:

```
import pandas as pd
df = pd.read_csv('my_file.csv') # Load from your computer
print(df.head())                # Show top 5 rows
print(df.shape)                 # Show (rows, columns) → e.g. (200, 6)
```

my_data.info()

```
my_data.info()
```

What it does: Prints a summary of the entire dataset — column names, how many values each column has, and the data type (int, float, object/text) of each column.

What .info() tells us for our drug dataset:

Column	Count	Data Type
Age	200	int64 (numbers — good, ready to use)
Sex	200	object (text: M/F — must convert to numbers)
BP	200	object (text: High/Low/Normal — must convert)
Cholesterol	200	object (text: High/Normal — must convert)
Na_to_K	200	float64 (decimal numbers — good, ready to use)
Drug	200	object (text: DrugA/B/C/X/Y — this is our TARGET)

Section 5: Data Preprocessing — Encoding Text to Numbers

The Decision Tree algorithm can only work with numbers. Text values like 'Male', 'High', 'Normal' need to be converted to numbers first. We use LabelEncoder for this.

```
label_encoder = LabelEncoder()
my_data['Sex'] = label_encoder.fit_transform(my_data['Sex'])
my_data['BP'] = label_encoder.fit_transform(my_data['BP'])
```

```
my_data['Cholesterol'] = label_encoder.fit_transform(my_data['Cholesterol'])
```

Line-by-Line Explanation:

label_encoder = LabelEncoder() — This creates a new LabelEncoder object and stores it in a variable called 'label_encoder'. Think of it as picking up a tool from the toolbox.



Syntax:

label_encoder.fit_transform(column) → Two steps in one:

fit → The encoder LOOKS at the column and figures out all unique values

transform → It then REPLACES each value with its corresponding number

my_data['Sex'] = ... — We are REPLACING the 'Sex' column in our table with the encoded (number) version. The ['Sex'] part selects just that column from the table.

What the encoding looks like:

BEFORE encoding: AFTER encoding:

Sex: M, F, F, M, M → Sex: 1, 0, 0, 1, 1

BP: High, Low, Normal → BP: 0, 1, 2

Cholesterol: High, Normal → Cholesterol: 0, 1

Rule: LabelEncoder assigns numbers alphabetically

F → 0, M → 1 | High → 0, Low → 1, Normal → 2

Another example to understand fit_transform:

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
cities = ['Delhi', 'Mumbai', 'Chennai', 'Delhi']
```

```
result = le.fit_transform(cities)
```

```
print(result) # Output: [1, 2, 0, 1]
```

Chennai → 0 (alphabetically first)

Delhi → 1

Mumbai → 2

Checking for Missing Values

```
my_data.isnull().sum()
```

What it does: Checks every column for missing (empty) values and shows the count. In our dataset, all columns show 0 — meaning there are no missing values. This is great news!

Understanding `isnull().sum()`:

`isnull()` → Returns True/False for each cell (True = missing, False = has value)

`.sum()` → Adds up all the Trues (each True = 1, False = 0)

If output shows Age: 5, it means 5 rows have missing Age values

If output shows Age: 0, it means NO missing values in Age — perfect!

Section 6: Exploring Correlations

```
custom_map = {'drugA':0, 'drugB':1, 'drugC':2, 'drugX':3, 'drugY':4}
my_data['Drug_num'] = my_data['Drug'].map(custom_map)

my_data.drop('Drug', axis=1).corr()['Drug_num']
```

custom_map — Creating a Dictionary



Syntax:

```
my_dict = { 'key1': value1, 'key2': value2, ... }
```

A dictionary in Python stores KEY → VALUE pairs.

Like a real dictionary: word → definition

What this does: We create a mapping from drug names (text) to numbers so we can calculate correlation. drugA → 0, drugB → 1, drugC → 2, etc.

`.map(custom_map)`

What it does: The `.map()` function goes through every row in the 'Drug' column and replaces each drug name with its corresponding number from our `custom_map` dictionary. The result is stored in a new column called 'Drug_num'.

Example of `.map()` in action:

```
custom_map = {'cat': 0, 'dog': 1, 'bird': 2}
pets = pd.Series(['dog', 'cat', 'bird', 'dog'])
print(pets.map(custom_map)) # Output: 1, 0, 2, 1
```

.drop('Drug', axis=1).corr()['Drug_num']

.drop('Drug', axis=1): Temporarily removes the 'Drug' column. axis=1 means we are dropping a COLUMN (axis=0 means dropping a ROW). We drop it because Drug_num already has the same information in number form.

.corr(): Calculates the correlation between all columns. Correlation is a number between -1 and +1 that tells us how closely two things move together.

How to read correlation values:

+1.0 = Perfect positive link (when one goes up, the other also goes up)

0.0 = No link at all

-1.0 = Perfect negative link (when one goes up, the other goes down)

From our dataset results:

Na_to_K and BP are most strongly correlated with Drug choice

Section 7: Visualizing the Data Distribution

```
category_counts = my_data['Drug'].value_counts()
print(category_counts)

plt.bar(category_counts.index, category_counts.values, color='blue')
plt.xlabel('Drug')
plt.ylabel('Count')
plt.title('Category Distribution')
plt.xticks(rotation=45)
plt.show()
```

Line-by-Line:

my_data['Drug'].value_counts(): Counts how many patients were given each drug. For example: drugY: 91, drugX: 54, drugA: 23, etc.



Syntax:

plt.bar(x_values, y_values, color='...')	→ Creates a bar chart
plt.xlabel('...')	→ Label for the X-axis (horizontal)
plt.ylabel('...')	→ Label for the Y-axis (vertical)
plt.title('...')	→ Title shown at the top of the chart
plt.xticks(rotation=45)	→ Rotate X labels 45 degrees (easier to read)
plt.show()	→ Display the chart

Simple bar chart example:

import matplotlib.pyplot as plt

```
subjects = ['Maths', 'Science', 'English']
scores = [85, 92, 78]
plt.bar(subjects, scores, color='green')
plt.title('Student Scores')
plt.ylabel('Marks')
plt.show()
```

Section 8: Splitting Data into Training and Testing Sets

```
y = my_data['Drug']
X = my_data.drop(['Drug', 'Drug_num'], axis=1)

X_trainset, X_testset, y_trainset, y_testset = train_test_split(X, y,
test_size=0.3, random_state=32)
```

Separating X (Features) and y (Target)

y = my_data['Drug']: This is our TARGET variable — what we want to predict. 'y' is just the standard name used in machine learning for the output/answer column.

X = my_data.drop(['Drug', 'Drug_num'], axis=1): This is our FEATURES — the input columns that the model will use to make predictions. We drop 'Drug' and 'Drug_num' because those are the answers — we can't cheat and give the model the answer while training!

X vs y explained simply:

X (Features / Input) → Age, Sex, BP, Cholesterol, Na_to_K

y (Target / Output) → Drug (what we want to predict)

Think of it like an exam:

X = the question | y = the correct answer

train_test_split()

Syntax:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=32)
```

Parameters:

X → the features table

y → the target column

test_size=0.3 → 30% of data goes to testing, 70% to training

random_state=32 → a fixed seed so the split is the same every time you run the code

Why random_state? Without it, Python picks a DIFFERENT random split every time. With random_state=32, the split is always identical — making results reproducible.

What the split looks like for 200 rows:

Total data: 200 rows

Training set (70%): 140 rows → Model learns from these

Testing set (30%): 60 rows → We check accuracy on these

The model NEVER sees the test set during training.

This is like hiding 60 exam questions and only revealing them after the model is trained.

Section 9: Building the Decision Tree Model

```
drugTree = DecisionTreeClassifier(criterion='entropy', max_depth=4)
drugTree.fit(X_trainset, y_trainset)
```

DecisionTreeClassifier()



Syntax:

DecisionTreeClassifier(criterion='...', max_depth=N)

criterion='entropy' → The method used to decide the BEST question to ask at each node

max_depth=4 → How many levels of questions the tree is allowed to have

What is Entropy?

Entropy measures how 'mixed up' or 'impure' a group is.

Example:

A bag with 100 red balls only → Entropy = 0 (pure, no confusion)

A bag with 50 red + 50 blue → Entropy = 1 (maximum confusion)

The Decision Tree always tries to ask questions that REDUCE entropy the most.

In other words: it asks the question that makes each resulting group as PURE as possible.

max_depth=4 means: only 4 levels of questions are allowed.

This prevents OVERFITTING (memorizing the training data instead of learning patterns).

drugTree.fit(X_trainset, y_trainset)



Syntax:

```
model.fit( X_train, y_train ) → TRAINS the model on training data
```

What happens: The Decision Tree looks at all 140 training rows and figures out the BEST questions to ask — in what order — to correctly identify each drug. After .fit() is done, the model has 'learned' the rules.

Analogy:

Calling .fit() is like a student reading a textbook.

After reading (fitting), the student (model) knows the rules.

Now they are ready to answer questions (predict on test data).

Section 10: Making Predictions and Evaluating Accuracy

```
tree_predictions = drugTree.predict(X_testset)
print('Decision Trees Accuracy: ', metrics.accuracy_score(y_testset,
tree_predictions))
```

drugTree.predict(X_testset)

 Syntax:

```
model.predict( X_test ) → Makes predictions on new/unseen data
```

What happens: The trained model now looks at the 60 test rows (which it has NEVER seen before) and predicts which drug should be given to each patient. The result is stored in 'tree_predictions' — a list of 60 predicted drug names.

metrics.accuracy_score(y_testset, tree_predictions)

 Syntax:

```
metrics.accuracy_score( actual_answers, predicted_answers )
```

Returns a decimal between 0 and 1:

1.0 = 100% accuracy (perfect)

0.5 = 50% accuracy (coin flip level)

0.0 = 0% accuracy (completely wrong)

Our result: The model gets 98.33% accuracy — meaning it correctly identified the right drug for 59 out of 60 test patients. Excellent!

Understanding accuracy_score:

```
actual    = ['DrugA', 'DrugY', 'DrugX', 'DrugB']
```

```
predicted = ['DrugA', 'DrugY', 'DrugX', 'DrugA'] # Last one wrong
```

```
accuracy = 3 correct / 4 total = 0.75 = 75%
```

Section 11: Visualizing the Decision Tree

```
plot_tree(drugTree)
plt.show()
```

What happens: This generates a visual diagram of our decision tree. Each box (node) shows: the question being asked, the entropy value, the number of samples at that node, and the final drug class (at leaf nodes).

How to read the Decision Tree diagram:

ROOT NODE (top box):

Shows the FIRST question the tree asks.
e.g., $\text{Na_to_K} \leq 14.627$

LEFT BRANCH → condition is TRUE

RIGHT BRANCH → condition is FALSE

LEAF NODES (bottom boxes, no children):

These are the final answers (drug classes).

From our trained tree — the decision rules:

Drug Y → $\text{Na_to_K} > 14.627$

Drug A → $\text{Na_to_K} \leq 14.627$, BP = High, Age ≤ 50.5

Drug B → $\text{Na_to_K} \leq 14.627$, BP = High, Age > 50.5

Drug C → $\text{Na_to_K} \leq 14.627$, BP = Low

Drug X → $\text{Na_to_K} \leq 14.627$, BP = Normal, Cholesterol = High

Section 12: Effect of Changing max_depth

The notebook asks: What happens if we reduce max_depth from 4 to 3?

```
drugTree = DecisionTreeClassifier(criterion='entropy', max_depth=3)
drugTree.fit(X_trainset, y_trainset)
tree_predictions = drugTree.predict(X_testset)
print('Decision Trees Accuracy: ', metrics.accuracy_score(y_testset,
tree_predictions))
```

Why would accuracy change?

max_depth=4 → Tree has 4 levels of questions → More specific rules → 98.33% accuracy

max_depth=3 → Tree has only 3 levels → Fewer rules → May be less accurate

This is the trade-off in machine learning:

Too shallow (depth=1, 2) → Underfitting → Model too simple, misses patterns

Too deep (depth=20+) → Overfitting → Model memorizes noise

Just right (depth=4 here) → Good balance → High accuracy on unseen data

Section 13: Complete Code Summary

Here is the entire notebook code in one place for reference:

```
# Step 1: Import libraries
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn import metrics
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline

# Step 2: Load data
path = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/.../drug200.csv'
my_data = pd.read_csv(path)

# Step 3: Encode text columns to numbers
label_encoder = LabelEncoder()
my_data['Sex'] = label_encoder.fit_transform(my_data['Sex'])
my_data['BP'] = label_encoder.fit_transform(my_data['BP'])
my_data['Cholesterol'] = label_encoder.fit_transform(my_data['Cholesterol'])

# Step 4: Check for missing values
my_data.isnull().sum()

# Step 5: Add Drug_num column for correlation
custom_map = {'drugA':0, 'drugB':1, 'drugC':2, 'drugX':3, 'drugY':4}
my_data['Drug_num'] = my_data['Drug'].map(custom_map)
my_data.drop('Drug', axis=1).corr()['Drug_num']

# Step 6: Visualize drug distribution
category_counts = my_data['Drug'].value_counts()
plt.bar(category_counts.index, category_counts.values, color='blue')
plt.title('Category Distribution')
plt.show()
```

```

# Step 7: Split into X and y
y = my_data['Drug']
X = my_data.drop(['Drug', 'Drug_num'], axis=1)

# Step 8: Train/Test split
X_trainset, X_testset, y_trainset, y_testset = train_test_split(X, y,
test_size=0.3, random_state=32)

# Step 9: Build and train the Decision Tree
drugTree = DecisionTreeClassifier(criterion='entropy', max_depth=4)
drugTree.fit(X_trainset, y_trainset)

# Step 10: Predict and evaluate
tree_predictions = drugTree.predict(X_testset)
print('Accuracy:', metrics.accuracy_score(y_testset, tree_predictions))

# Step 11: Visualize the tree
plot_tree(drugTree)
plt.show()

```

Quick Reference Card — Functions Used in This Notebook

Function / Method	What It Does	Example
<code>pd.read_csv()</code>	Loads a CSV file into a table	<code>df = pd.read_csv('data.csv')</code>
<code>df.info()</code>	Shows column types and counts	<code>my_data.info()</code>
<code>df.isnull().sum()</code>	Counts missing values	<code>my_data.isnull().sum()</code>
<code>LabelEncoder().fit_transform()</code>	Converts text to numbers	<code>le.fit_transform(df['Sex'])</code>
<code>df['col'].map(dict)</code>	Replaces values using a dictionary	<code>df['Drug'].map({'A':0,'B':1})</code>
<code>df.drop('col', axis=1)</code>	Removes a column	<code>df.drop('Drug', axis=1)</code>
<code>df.corr()</code>	Computes correlation between columns	<code>df.corr()['target_col']</code>
<code>df['col'].value_counts()</code>	Counts occurrences of each value	<code>my_data['Drug'].value_counts()</code>
<code>train_test_split()</code>	Splits data into train/test	<code>X_train,X_test,y_train,y_test = ...</code>
<code>model.fit(X, y)</code>	Trains the model	<code>drugTree.fit(X_trainset, y_trainset)</code>
<code>model.predict(X)</code>	Makes predictions	<code>drugTree.predict(X_testset)</code>
<code>accuracy_score(y, pred)</code>	Measures accuracy (0 to 1)	<code>metrics.accuracy_score(y_test, pred)</code>
<code>plot_tree(model)</code>	Draws the decision tree	<code>plot_tree(drugTree)</code>

Summary — What Did We Learn Today?

- A Decision Tree works like a flowchart of YES/NO questions to reach a final answer.
- We used Python libraries: NumPy (maths), Pandas (tables), Matplotlib (charts), and Scikit-learn (ML).
- Text columns (Sex, BP, Cholesterol) must be converted to numbers using LabelEncoder before using them in ML.
- We split data into 70% training and 30% testing to fairly measure model performance.
- DecisionTreeClassifier with criterion='entropy' and max_depth=4 gives us 98.33% accuracy.
- We visualized the tree using plot_tree() to understand the rules the model learned.
- max_depth controls how deep (complex) the tree can grow — too shallow = underfitting, too deep = overfitting.

Great work! You now understand Decision Trees from the ground up.